Effective Computation of a Feedback Arc Set Using Machine Learning-Enhanced PageRank Batching

Georgios Papadakis, Dr. Panagiotis Lionakis, Dr. Michalis Mountantonakis

Computer Science Department, University of Crete, Greece

October 2026

Abstract

A Feedback Arc Set (FAS) of a directed graph is a set of arcs whose removal makes the graph acyclic. Finding a minimum-cardinality FAS is an NP-hard combinatorial optimization problem with important applications in hierarchical graph drawing and the analysis of cyclic dependencies [11, 16, 7]. PageRankFAS is a recent heuristic that decomposes a graph into Strongly Connected Components (SCCs), constructs the corresponding directed line digraphs, and applies undamped PageRank to rank the original arcs according to their structural participation in cycles [17, 12, 9]. Although PageRankFAS produces high-quality solutions, it removes only one selected arc from each component per iteration. The SCC decomposition, line digraph, and PageRank scores must therefore be recomputed repeatedly as the graph changes, creating a scalability bottleneck on large or dense instances [9].

This thesis presents **ML-Batching-PageRankFAS**, an extension that seeks to reduce this recomputation cost by removing multiple candidate arcs during each iteration. For the top-K PageRank-ranked arcs of an SCC, the method represents every candidate pair using seven structural and score-based features. A class-balanced Random Forest classifier estimates whether removing a pair concurrently is likely to increase the final FAS cardinality relative to the sequential baseline [5, 13]. These predictions define an undirected conflict graph, and an approximate maximum independent set selects a batch containing no pair classified as conflicting at the chosen threshold [3, 20]. The resulting method preserves the iterative structure of PageRankFAS while amortizing line-digraph construction and PageRank evaluation across several removals. Its evaluation compares runtime and FAS cardinality with the baseline method on synthetic directed graphs, tournament instances, and real directed networks.

1 Introduction

Directed graphs provide a natural representation of systems in which relationships have an orientation. A directed graph is written as $G = (V, E)$, where V is the set of vertices and $E \subseteq V \times V$ is the set of directed edges, also called arcs. A directed cycle is a closed directed path, while a Directed Acyclic Graph (DAG) contains no directed cycles. DAGs admit a topological ordering and are therefore useful for representing hierarchical structures and dependency relationships [7, 9].

When a directed graph contains cycles, one way to make it acyclic is to remove at least one arc from every directed cycle. A *Feedback Arc Set* (FAS) is a subset $F \subseteq E$ such that

$$G' = (V, E \setminus F)$$

is a DAG. The *Minimum Feedback Arc Set problem* asks for a valid set F of minimum cardinality. Its decision version is NP-complete, making the corresponding optimization problem NP-hard [11, 10]. The problem is also complementary to the Maximum Acyclic Subgraph problem: minimizing the number of removed arcs is equivalent to maximizing the number of arcs retained in an acyclic subgraph [2].

The FAS problem is particularly important in hierarchical graph drawing. In the Sugiyama framework, cycle removal is the first phase used to obtain an acyclic orientation before assigning vertices to layers and reducing crossings [16, 7]. The arcs selected during this phase may be reversed rather than permanently deleted, so smaller feedback arc sets generally preserve more of the original edge

orientation. Feedback-arc computation also appears in process-system decomposition, where cyclic dependencies must be broken to obtain a feasible evaluation order [1].

Because exact computation is difficult, practical methods usually rely on heuristics. The GreedyFAS algorithm constructs a linear ordering by repeatedly processing sinks, sources, and vertices with a favorable difference between out-degree and in-degree [8]. SortFAS instead uses an insertion-based ordering strategy designed to reduce the number of backward arcs [4, 15]. These algorithms are computationally efficient and have been evaluated on large directed networks, but their local ordering decisions may produce feedback arc sets larger than necessary [15, 9]. Exact approaches based on integer programming and lazy cycle generation can establish optimality on suitable instances, but their scalability is constrained by the potentially exponential number of simple directed cycles [1].

PageRankFAS was introduced to improve solution quality through an edge-ranking strategy based on PageRank [9]. The algorithm first decomposes the input graph into its Strongly Connected Components (SCCs). Every directed cycle is contained entirely within one SCC, so acyclic and single-vertex components can be excluded from further processing. SCC decomposition can be performed in $O(|V| + |E|)$ time using Tarjan's algorithm [17].

For each non-trivial SCC, PageRankFAS constructs a directed line graph, or line digraph. In this representation, every arc of the original component becomes a vertex, and two line-graph vertices are connected whenever the corresponding original arcs form a directed path of length two. Consequently, cycles in the original graph are represented by cycles in the line digraph [9, 14]. PageRank is then applied without teleportation to assign a score to each line-graph vertex and therefore to each original arc [6, 12, 9]. After a fixed number of iterations, the highest-scoring arc of each processed component is selected for removal. The procedure repeats until the graph is acyclic.

Experimental results reported for PageRankFAS show that it can produce substantially smaller feedback arc sets than established heuristics on graph-drawing instances and can improve previously reported solutions for some large web graphs [9]. This improvement, however, comes with a computational cost. Removing an arc changes paths, degrees, cycles, and possibly the SCC decomposition itself. The previous line digraph and PageRank scores can therefore no longer be reused directly. PageRankFAS repeatedly reconstructs the affected structures and recomputes the scores, even though only one arc per component is selected in each pass.

The line digraph of a directed graph has $|E|$ vertices and

$$\sum_{v \in V} d^-(v) d^+(v)$$

arcs, because every incoming arc of a vertex can be followed by each of its outgoing arcs. Its construction and traversal therefore require work proportional to

$$O\left(|E| + \sum_{v \in V} d^-(v) d^+(v) \right)$$

for a single pass [9]. Repeating this computation after individual removals can dominate execution time when SCCs are large or dense. The central question of this thesis is therefore whether several high-ranking arcs can be removed together while limiting the deterioration in FAS cardinality that may result from redundant removals.

A direct top-K batching rule is not sufficient. Two highly ranked arcs may participate in the same cycle or closely related cyclic structures. Once one of them is removed, deleting the other may no longer be necessary. Conversely, arcs acting on different cyclic regions may be removed in the same iteration without this form of redundancy. The proposed **ML-Batching-PageRankFAS** method treats this interaction as a supervised pairwise classification problem.

At every iteration, the method obtains the top-K PageRank-ranked candidate arcs from each non-trivial SCC. For every candidate pair, it extracts seven features describing shared endpoints, directed reachability, common predecessor and successor neighborhoods, and the relationship between the two

PageRank scores. Training labels are generated by comparing the FAS cardinality obtained after removing the pair with the cardinality obtained by the sequential PageRankFAS baseline. A Random Forest classifier with balanced class weights is then trained to estimate pairwise conflict probabilities [5, 13].

The predicted probabilities are converted into an undirected conflict graph. Its vertices represent candidate arcs, and an edge connects two candidates when their estimated conflict probability reaches a configurable threshold τ. An approximation of a maximum independent set is used to select a subset containing no predicted pairwise conflicts [3]. All arcs in this subset are removed in one batch, after which the SCCs and PageRank scores are recomputed. The approach therefore retains the adaptive, iterative behavior of PageRankFAS while attempting to distribute the expensive line-digraph computation over several arc removals.

The main contributions of this thesis are: the formulation of pairwise conflict prediction for PageRank-based FAS batching; the design of a seven-feature representation for candidate-arc interactions; a reproducible pipeline for generating labeled training pairs and fitting a class-balanced Random Forest model; the construction of a conflict graph for selecting a predicted conflict-free batch; and an experimental framework for evaluating the trade-off between execution time and FAS cardinality on synthetic graphs, tournament instances, and real directed networks.

The remainder of the thesis is organized as follows. Section 2 presents the FAS problem and the PageRankFAS baseline, including SCC decomposition, directed line graphs, PageRank scoring, and the recomputation bottleneck. Section 3 describes ML-Batching-PageRankFAS, its training-data generation, feature representation, classifier, conflict graph, and batching procedure. Section 4 presents the experimental methodology and evaluates runtime and solution quality. Section 5 summarizes the findings, discusses limitations, and identifies directions for future work.

2 Existing Algorithm: PageRankFAS

This section presents the PageRankFAS heuristic introduced by Geladaris, Lionakis, and Tollis [9]. PageRankFAS approaches the Feedback Arc Set problem by transforming arcs of the input digraph into vertices of a directed line graph and then applying PageRank to obtain an arc-ranking heuristic. The algorithm repeatedly removes a highly ranked arc from each cyclic strongly connected component until the remaining graph is acyclic.

The presentation first introduces the required graph-theoretic definitions, then describes strongly connected component decomposition, directed line graphs, the PageRank scoring procedure, and finally the complete iterative algorithm. The description of the published method is followed by a short implementation note identifying the defensive choices made in the accompanying Python baseline.

2.1 Problem definition

Let $G = (V, E)$ be a finite directed graph, or digraph, where V is the set of vertices and $E \subseteq V \times V$ is the set of directed edges, also called arcs. A directed cycle is a closed directed path whose internal vertices are distinct. A directed graph containing no directed cycle is called a directed acyclic graph (DAG) [21].

A *feedback arc set* (FAS) of G is a subset $F \subseteq E$ such that

$$G - F = (V, E \setminus F)$$

is acyclic. The minimum feedback arc set problem asks for a minimum-cardinality set

$$F^\star \in \arg\min_{F \subseteq E} \{|F| \mid (V, E \setminus F) \text{ is a DAG}\}. \tag{1}$$

The corresponding decision problem is NP-complete, and therefore the optimization problem is NP-hard [11, 10]. This computational difficulty motivates the use of heuristics such as GreedyFAS, SortFAS, and PageRankFAS [8, 4, 15, 9].

The minimum FAS problem is complementary to the maximum acyclic subgraph problem. For any $F \subseteq E$, the set F is a FAS if and only if $E \setminus F$ induces an acyclic spanning subgraph. Consequently, minimizing $|F|$ is equivalent to maximizing the number of retained arcs [2].

It is important to distinguish a *minimal* FAS from a *minimum* FAS. A FAS is minimal when none of its proper subsets is also a FAS. A minimum FAS has the smallest cardinality among all possible feedback arc sets. Every minimum FAS is minimal, but a minimal FAS need not have minimum cardinality [21].

2.2 Restriction to strongly connected components

A digraph is *strongly connected* if, for every ordered pair of vertices u, v, there is a directed path from u to v. A strongly connected component (SCC) is a maximal vertex-induced strongly connected subgraph. SCCs can be computed in $O(|V| + |E|)$ time using a depth-first-search-based algorithm [17].

Every directed cycle is contained entirely within one SCC. Equivalently, contracting every SCC to a single vertex produces an acyclic condensation graph. It is therefore sufficient to search for feedback arcs independently inside the cyclic SCCs of G [17, 9].

After self-loops have been removed, an SCC containing only one vertex cannot contain a directed cycle and may be omitted. PageRankFAS therefore operates only on non-trivial SCCs. This decomposition is recomputed after arc removals because deleting an arc can split one SCC into several smaller components.

2.3 Directed line graph

For a digraph $H = (V_H, E_H)$, its directed line graph, or *line digraph*, is denoted by

$$L(H) = (V_L, E_L).$$

Each vertex of $L(H)$ represents one arc of H:

$$V_L = E_H. \tag{2}$$

Two line-graph vertices are adjacent when their corresponding arcs are consecutive in H. Formally,

$$E_L = \left\{ \big((u,v),(v,w)\big) \;\middle|\; (u,v) \in E_H,\ (v,w) \in E_H \right\}. \tag{3}$$

Thus, every directed edge of $L(H)$ represents a directed walk of length two in H. A directed cycle

$$v_1 \to v_2 \to \cdots \to v_k \to v_1$$

in H induces the directed cycle

$$(v_1, v_2) \to (v_2, v_3) \to \cdots \to (v_k, v_1) \to (v_1, v_2)$$

in $L(H)$. This correspondence allows a vertex-ranking method on the line digraph to serve as an arc-ranking method on the original digraph [9].

The line digraph has exactly $|E_H|$ vertices. For each $v \in V_H$, every incoming arc of v may be followed by every outgoing arc of v. Hence,

$$|E_L| = \sum_{v \in V_H} d_H^-(v) d_H^+(v), \tag{4}$$

where $d_H^-(v)$ and $d_H^+(v)$ denote the in-degree and out-degree of v, respectively. The size of the line digraph is therefore

$$|V_L| + |E_L| = |E_H| + \sum_{v \in V_H} d_H^-(v) d_H^+(v). \tag{5}$$

This degree-product term explains why constructing and processing the line digraph becomes costly for dense SCCs or for SCCs containing high-degree vertices [9].

Algorithm 1 gives a direct construction. The accompanying implementation uses NetworkX's directed `line_graph` operation, which implements the same adjacency definition [22, 18].

Algorithm 1: BUILDLINEDIGRAPH

Input: A directed graph $H = (V_H, E_H)$
Output: The directed line graph $L(H) = (V_L, E_L)$

```
V_L ← E_H
E_L ← ∅
foreach v ∈ V_H do
    foreach (u, v) ∈ E_H do
        foreach (v, w) ∈ E_H do
            E_L ← E_L ∪ {((u, v), (v, w))}
return (V_L, E_L)
```

2.4 PageRank-based arc scoring

PageRank was introduced as a method for assigning importance scores to vertices in a directed hyperlink graph [6, 12]. PageRankFAS applies the same propagation principle to the vertices of the line digraph. Since each vertex of $L(H)$ represents an arc of H, the resulting value $p(e)$ is interpreted as a score for the original arc e.

Let $N = |V_L|$. PageRankFAS initializes every line-graph vertex uniformly:

$$p^{(0)}(e) = \frac{1}{N}, \qquad e \in V_L. \tag{6}$$

For iteration $t \geq 1$, the score of $e' \in V_L$ is updated by

$$p^{(t)}(e') = \sum_{e \in \mathrm{Pred}_L(e')} \frac{p^{(t-1)}(e)}{d_L^+(e)}, \tag{7}$$

where $\mathrm{Pred}_L(e')$ is the set of predecessors of e' in $L(H)$, and $d_L^+(e)$ is the out-degree of e in the line digraph.

Unlike conventional Web PageRank, this update contains no teleportation term. In other words, PageRankFAS uses damping parameter $\alpha = 1$. The algorithm applies the update inside line digraphs derived from strongly connected components, where the random walk remains within the component [9]. Geladaris et al. report that the rankings obtained for their experiments stabilized sufficiently after five iterations, and PageRankFAS therefore uses the fixed value $k = 5$ [9]. This is an empirical algorithm parameter rather than a proof that the exact stationary distribution is reached after five steps.

The score is used as a heuristic indicator of an arc's participation in cyclic structure. The algorithm selects an arc with maximum score, but this choice is not claimed to maximize the exact number of cycles destroyed, nor does it guarantee that the final FAS has minimum cardinality.

Algorithm 2: LINEDIGRAPHPAGERANK

Input: A line digraph $L = (V_L, E_L)$; iteration count k
Output: A score $p(e)$ for every $e \in V_L$

$N \leftarrow |V_L|$
foreach $e \in V_L$ **do**
  $p(e) \leftarrow 1/N$
for $t \leftarrow 1$ **to** k **do**
  foreach $e \in V_L$ **do**
    $q(e) \leftarrow 0$
  foreach $e \in V_L$ **do**
    if $d_L^+(e) > 0$ **then**
      foreach $z \in \mathrm{Succ}_L(e)$ **do**
        $q(z) \leftarrow q(z) + p(e)/d_L^+(e)$
    else
      ▷ `Defensive dangling-node handling`
      foreach $z \in V_L$ **do**
        $q(z) \leftarrow q(z) + p(e)/N$
  $p \leftarrow q$
return p

The dangling-node branch in Algorithm 2 records the behavior of the accompanying Python implementation. It redistributes the score of a zero-out-degree line-graph vertex uniformly over V_L. Such a vertex is not expected when the routine is applied to the line digraph of a non-trivial SCC, but the branch makes the implementation robust if the scoring routine is reused on a more general digraph.

2.5 The PageRankFAS procedure

PageRankFAS maintains a working copy of the input graph and an initially empty set F. Any self-loop (v, v) is necessarily a cycle of length one. It must therefore belong to every valid FAS and can be removed during preprocessing.

The algorithm then repeats the following operations until the working graph is acyclic:

1. compute the SCC decomposition of the current graph;
2. discard singleton components, since self-loops have already been removed;
3. construct the line digraph of each remaining SCC;
4. perform five undamped PageRank iterations on that line digraph;
5. select one maximum-scoring line-graph vertex, interpret it as an arc of the original graph, and remove that arc.

One selected arc is removed from each non-trivial SCC found at the beginning of the current global pass. After the pass, SCCs and scores are recomputed because the removals may have changed both reachability and component structure. The complete procedure is shown in Algorithm 3.

Algorithm 3: PAGERANKFAS

Input: A directed graph $G = (V, E)$; PageRank iteration count $k = 5$
Output: A feedback arc set $F \subseteq E$

$H \leftarrow G$
$F \leftarrow \varnothing$
$E_{\text{self}} \leftarrow \{(v, v) \in E(H)\}$
$F \leftarrow F \cup E_{\text{self}}$
$H \leftarrow H - E_{\text{self}}$
while *H is not a DAG* **do**
    $\mathscr{C} \leftarrow \text{STRONGLYCONNECTEDCOMPONENTS}(H)$
    foreach *$C \in \mathscr{C}$ such that $|V(C)| > 1$* **do**
        $L_C \leftarrow \text{BUILDLINEDIGRAPH}(C)$
        if $|V(L_C)| > 0$ **then**
            $p \leftarrow \text{LINEDIGRAPHPAGERANK}(L_C, k)$
            choose $e^\star \in \arg\max_{e \in V(L_C)} p(e)$
            $F \leftarrow F \cup \{e^\star\}$
            remove $e^\star$ from H
return F

Termination follows because every execution of the body removes at least one arc from a cyclic graph. When the loop terminates, H is a DAG and $H = G - F$; therefore the returned set F is a valid feedback arc set. This establishes validity, but not minimum cardinality.

When several arcs share the same maximum score, the mathematical algorithm may choose any maximizer. In a concrete implementation, the selected arc can consequently depend on the iteration order of the line-graph vertices unless an explicit deterministic tie-breaking rule is introduced.

2.6 Computational cost and recomputation bottleneck

Consider one processed SCC $C = (V_C, E_C)$. Its line digraph contains

$$|E_C|$$

vertices and

$$\sum_{v \in V_C} d_C^-(v) d_C^+(v)$$

arcs. Constructing the line digraph and performing a constant number $k = 5$ of PageRank iterations therefore requires time proportional to

$$O\left(|E_C| + \sum_{v \in V_C} d_C^-(v) d_C^+(v)\right). \tag{8}$$

The SCC decomposition of the current graph additionally costs $O(|V| + |E|)$ per global pass [17].

The principal scalability limitation is not only the cost of one line digraph, but the repeated reconstruction of these structures. PageRankFAS removes only one selected arc from each processed SCC in a pass. An arc removal can change directed paths, split an SCC, and alter the line digraph and its PageRank scores. Consequently, the algorithm recomputes the SCC decomposition, reconstructs the affected line digraphs, and repeats the PageRank iterations in subsequent passes [9].

The number and sizes of the SCCs may change throughout execution, so a single tight total-runtime expression does not follow merely by multiplying Equation (8) by the final FAS size. Nevertheless, the repeated line-digraph construction is the dominant practical bottleneck identified in the original study, especially when the cyclic components are large or dense [9]. This repeated-computation pattern motivates the batch-removal method introduced in Section 3.

3 Our Approach: ML-Batching-PageRankFAS

3.1 Motivation and design objective

As discussed in Section 2.6, PageRankFAS repeatedly decomposes the current graph into strongly connected components (SCCs), constructs a line digraph for each cyclic component, executes five undamped PageRank iterations, and removes one maximum-scoring arc from each component [9]. Removing an arc may change the SCC decomposition, the line digraph, and the PageRank ordering. These structures must consequently be reconstructed during the next global pass.

ML-Batching-PageRankFAS preserves this iterative structure but attempts to remove more than one candidate arc from each processed SCC. The objective is to amortize the cost of line-digraph construction and PageRank propagation over a batch of removals. A batch must nevertheless be chosen conservatively: removing several high-ranked arcs without accounting for their interaction can select redundant arcs and increase the final FAS cardinality.

Let $C_K(S)$ denote the K highest-ranked arcs of a non-trivial SCC S, according to the five-step PageRank scores of $L(S)$. The proposed method learns a pairwise function

$$\widehat{q}(e_i, e_j) = \widehat{\Pr}(y(e_i, e_j) = 1 \mid \phi(e_i, e_j)), \tag{9}$$

where $y = 1$ denotes an empirically defined conflict and $\phi \in \mathbb{R}^7$ is a feature vector describing the relationship between two candidate arcs. The implementation uses a Random Forest classifier for this purpose [5, 13].

The learned value in Equation (9) is a model score returned by `predict_proba`; it should not be interpreted as a formally calibrated probability because the current training pipeline does not perform probability calibration [25]. Given a decision threshold τ, a pair is treated as conflicting when $\widehat{q}(e_i, e_j) \geq \tau$.

The pairwise predictions induce an undirected conflict graph. An approximate maximum independent set of this graph is then removed as one batch. This set is free of *predicted* pairwise conflicts at threshold τ, but it is not guaranteed to be free of true conflicts. The method is therefore a heuristic extension of PageRankFAS rather than an exact or quality-preserving algorithm.

3.2 Candidate extraction within each SCC

Let H be the current working graph. As in PageRankFAS, self-loops are first added to the output set and removed from H, since every self-loop forms a directed cycle of length one. While H remains cyclic, the algorithm computes its non-trivial SCCs. Every directed cycle is contained within an SCC, so different SCCs can be processed independently during the current pass [17, 9].

For every non-trivial SCC S, the algorithm constructs the directed line graph $L(S)$ and invokes the same five-step undamped PageRank computation used by the baseline. If $p_S(e)$ is the resulting score of the line-graph vertex corresponding to arc $e \in E(S)$, the candidate sequence is

$$C_K(S) = \left(e_{(1)}, e_{(2)}, \ldots, e_{(r)}\right), \qquad r = \min\{K, |E(S)|\}, \tag{10}$$

where

$$p_S(e_{(1)}) \geq p_S(e_{(2)}) \geq \cdots \geq p_S(e_{(r)}). \tag{11}$$

The implementation obtains this sequence by sorting the complete PageRank mapping in descending order and retaining its first K entries. The default inference value is $K = 20$. If $r = 1$, the only candidate is removed immediately, reproducing the single-arc behavior of PageRankFAS for that component.

Candidate selection is deliberately performed separately for every SCC. Consequently, conflict prediction is also restricted to pairs belonging to the same component during inference.

3.3 Pairwise feature representation

For each unordered pair $\{e_1, e_2\} \subseteq C_K(S)$, the implementation first imposes a consistent order. The higher-scoring arc is denoted by

$$e_A = (u, v),$$

and the other by

$$e_B = (x, y),$$

so that $p_S(e_A) \geq p_S(e_B)$. The feature extractor then returns

$$\phi(e_A, e_B) = (\phi_1, \phi_2, \phi_3, \phi_4, \phi_5, \phi_6, \phi_7). \tag{12}$$

The seven implemented features are as follows.

Shared endpoints. The first feature counts the endpoints appearing in both arcs:

$$\phi_1 = |\{u, v\} \cap \{x, y\}|. \tag{13}$$

For two distinct arcs in a simple digraph, this value is normally zero or one. The implementation nevertheless admits the general range $\{0, 1, 2\}$.

Directed distance from e_A to e_B. The second feature is the shortest-path distance from the head of e_A to the tail of e_B:

$$\phi_2 = \begin{cases} \mathrm{dist}_H(v, x), & \text{if } x \text{ is reachable from } v, \\ |V(H)|, & \text{otherwise.} \end{cases} \tag{14}$$

Directed distance from e_B to e_A. The reverse-oriented distance is

$$\phi_3 = \begin{cases} \mathrm{dist}_H(y, u), & \text{if } u \text{ is reachable from } y, \\ |V(H)|, & \text{otherwise.} \end{cases} \tag{15}$$

The graph order is used as a finite sentinel when no path exists; the directed distances are computed with NetworkX shortest-path routines [23, 18]. These two features record directed proximity in both orientations. They are indicators of possible cyclic interaction, but the implementation does not enumerate cycles and does not prove that a particular pair is redundant.

Common successors. Let

$$\mathrm{Succ}_H(z) = \{w \in V(H) : (z, w) \in E(H)\}.$$

The fourth feature is

$$\phi_4 = |\mathrm{Succ}_H(v) \cap \mathrm{Succ}_H(y)|. \tag{16}$$

Common predecessors. Similarly, let

$$\mathrm{Pred}_H(z) = \{w \in V(H) : (w, z) \in E(H)\}.$$

The fifth feature is

$$\phi_5 = |\mathrm{Pred}_H(u) \cap \mathrm{Pred}_H(x)|. \tag{17}$$

PageRank-score difference. The sixth feature measures the absolute score separation:

$$\phi_6 = |p_S(e_A) - p_S(e_B)|. \tag{18}$$

Because the pair is ordered by score, this is equal to $p_S(e_A) - p_S(e_B)$.

PageRank-score ratio. The final feature is

$$\phi_7 = \begin{cases} \dfrac{\min\{p_S(e_A), p_S(e_B)\}}{\max\{p_S(e_A), p_S(e_B)\}}, & \text{if } \max\{p_S(e_A), p_S(e_B)\} > 0, \\ 0, & \text{otherwise.} \end{cases} \tag{19}$$

This ratio lies in $[0,1]$ and expresses score similarity independently of the absolute score scale.

Algorithm 4 presents the complete feature construction in the same notation as the implementation.

Algorithm 4: PAIRWISEFEATURES

Input: Working digraph H; candidate arcs e_1, e_2; scores $p(e_1), p(e_2)$
Output: Feature vector $\phi(e_1, e_2) \in \mathbb{R}^7$

if $p(e_1) \geq p(e_2)$ **then**
  $e_A \leftarrow e_1$, $e_B \leftarrow e_2$
else
  $e_A \leftarrow e_2$, $e_B \leftarrow e_1$
$(u,v) \leftarrow e_A$, $(x,y) \leftarrow e_B$
$n \leftarrow |V(H)|$
$\phi_1 \leftarrow |\{u,v\} \cap \{x,y\}|$
if *x is reachable from v in H* **then**
  $\phi_2 \leftarrow \mathrm{dist}_H(v,x)$
else
  $\phi_2 \leftarrow n$
if *u is reachable from y in H* **then**
  $\phi_3 \leftarrow \mathrm{dist}_H(y,u)$
else
  $\phi_3 \leftarrow n$
$\phi_4 \leftarrow |\mathrm{Succ}_H(v) \cap \mathrm{Succ}_H(y)|$
$\phi_5 \leftarrow |\mathrm{Pred}_H(u) \cap \mathrm{Pred}_H(x)|$
$\phi_6 \leftarrow |p(e_A) - p(e_B)|$
if $\max\{p(e_A), p(e_B)\} > 0$ **then**
  $\phi_7 \leftarrow \min\{p(e_A), p(e_B)\} / \max\{p(e_A), p(e_B)\}$
else
  $\phi_7 \leftarrow 0$
return $(\phi_1, \phi_2, \phi_3, \phi_4, \phi_5, \phi_6, \phi_7)$

3.4 Training-label generation

The classifier is trained on synthetic directed graphs generated with an independent-edge $G(n,p)$-style model [19]. In the current implementation, each graph has

$$n \sim \mathrm{UniformInteger}(30, 100)$$

vertices and edge probability

$$p \sim \mathrm{Uniform}(0.05, 0.15).$$

Self-loops are removed, and generated DAGs are skipped. The default script attempts to generate $M = 3000$ graphs and processes them in parallel.

For each retained graph G, the training script first computes

$$b(G) = |\textsc{PageRankFAS}(G)|, \tag{20}$$

where PAGERANKFAS is the implemented heuristic baseline, not an exact minimum-FAS solver.

The script then performs one PageRank-scoring pass on every non-trivial SCC, combines the resulting arc scores, and retains the ten highest-scoring arcs over the entire graph. Let this training candidate set be $C_{10}^{\text{train}}(G)$. For every pair $\{e_A, e_B\} \subseteq C_{10}^{\text{train}}(G)$, it constructs

$$G_{A,B} = G - \{e_A, e_B\}$$

and evaluates

$$b_{A,B}(G) = 2 + |\textsc{PageRankFAS}(G_{A,B})|. \tag{21}$$

The corresponding label is

$$y(e_A, e_B) = \begin{cases} 1, & \text{if } b_{A,B}(G) > b(G), \\ 0, & \text{otherwise.} \end{cases} \tag{22}$$

Thus, label 1 means that forcing both arcs into the solution produced a larger FAS than the unmodified PageRankFAS run on the same graph. Label 0 means that the forced-pair result was no larger. This is an empirical label relative to one execution of the heuristic baseline. It is not a statement about the true minimum feedback arc set.

Algorithm 5: GENERATEPAIRWISETRAININGDATA

Input: Number of attempted graphs M; training candidate bound $K_{\text{train}} = 10$
Output: A dataset $\mathscr{D}$ of feature vectors and binary labels

$\mathscr{D} \leftarrow \varnothing$
for $i \leftarrow 1$ **to** M **do**
  sample n uniformly from $\{30, \ldots, 100\}$
  sample p uniformly from $[0.05, 0.15]$
  generate a directed independent-edge graph G with parameters n, p
  remove every self-loop from G
  if *G is a DAG* **then**
    continue
  $b \leftarrow |\textsc{PageRankFAS}(G)|$
  $C \leftarrow \textsc{GlobalTopKCandidates}(G, K_{\text{train}})$
  if $|C| < 2$ **then**
    continue
  foreach *unordered pair* $\{e_1, e_2\} \subseteq C$ **do**
    $\phi \leftarrow \textsc{PairwiseFeatures}(G, e_1, e_2, p(e_1), p(e_2))$
    $G' \leftarrow G - \{e_1, e_2\}$
    $b' \leftarrow 2 + |\textsc{PageRankFAS}(G')|$
    if $b' > b$ **then**
      $y \leftarrow 1$
    else
      $y \leftarrow 0$
    $\mathscr{D} \leftarrow \mathscr{D} \cup \{(\phi, y)\}$
return $\mathscr{D}$

3.5 Random Forest training and decision threshold

The generated examples are divided into an 80% training partition and a 20% test partition using a stratified split with random state 42. Stratification preserves the observed label proportions in both partitions. The implementation trains a Random Forest with 100 trees, random state 42, parallel tree construction, and `class_weight="balanced"` [5, 13].

For binary labels, balanced class weighting assigns class c the weight

$$w_c = \frac{N}{2N_c}, \tag{23}$$

where N is the total number of training examples and N_c is the number of examples belonging to class c. This is the balanced weighting rule implemented by scikit-learn [24]. Its purpose is to prevent the more frequent class from dominating the learned splitting criteria.

The training script evaluates the held-out test partition at thresholds

$$\tau \in \{0.50, 0.40, 0.35, 0.30\}$$

and reports accuracy, precision, recall, F_1, the confusion matrix, false negatives, and true positives. It also reports impurity-based feature importances and serializes the fitted model.

The threshold controls a structural trade-off. Lowering τ classifies more pairs as conflicts, inserts more edges into the conflict graph, and generally produces smaller batches. Increasing τ classifies fewer pairs as conflicts and can produce larger batches. The resulting relationship with runtime and FAS cardinality is empirical and must be evaluated experimentally; it is not guaranteed solely by the threshold definition.

In the reported implementation and experiments, the operational conflict threshold is fixed at $\tau = 0.70$. Thus, a candidate pair is represented by an edge in the conflict graph when its predicted conflict score is at least 0.70.

3.6 Conflict graph and batch selection

For an inference candidate set $C_K(S)$, define the undirected conflict graph

$$G_{\text{conf}} = (V_{\text{conf}}, E_{\text{conf}}), \tag{24}$$

where

$$V_{\text{conf}} = C_K(S) \tag{25}$$

and

$$E_{\text{conf}} = \left\{ \{e_i, e_j\} \subseteq C_K(S) : \widehat{q}(e_i, e_j) \geq \tau \right\}. \tag{26}$$

An independent set $I \subseteq V_{\text{conf}}$ contains no adjacent vertices. Therefore,

$$\widehat{q}(e_i, e_j) < \tau \qquad \text{for all distinct } e_i, e_j \in I. \tag{27}$$

Accordingly, an independent set represents a batch with no pair that the classifier marks as conflicting at threshold τ.

Finding a maximum independent set is NP-hard [11]. The implementation therefore invokes NetworkX's approximate `maximum_independent_set` routine, which is based on the approximation method of Boppana and Halldórsson [3, 20]. The returned batch should consequently be described as an *approximate maximum independent set*, not as an exact maximum independent set.

The implementation also contains a defensive fallback: if the approximation routine returns an empty set, the highest-ranked candidate is removed. Since the conflict graph is constructed with at least one vertex whenever this branch is reached, a non-empty independent set normally exists. The fallback nevertheless ensures progress if the selection routine produces an unexpected empty result.

3.7 Complete ML-Batching-PageRankFAS procedure

Algorithm 6 follows the implementation in `src/batching.py`. The graph is copied so that the input object is not modified. The trained model is loaded once, and all prediction calls within the algorithm use this fixed model.

Algorithm 6: ML-BATCHING-PAGERANKFAS

Input: Directed graph $G = (V, E)$; trained classifier $\mathscr{M}$; candidate bound $K = 20$; conflict threshold $\tau = 0.70$; PageRank iteration count $k = 5$
Output: A feedback arc set $F \subseteq E$

$H \leftarrow G$
$F \leftarrow \varnothing$
$E_{\text{self}} \leftarrow \{(v, v) \in E(H)\}$
$F \leftarrow F \cup E_{\text{self}}$
$H \leftarrow H - E_{\text{self}}$
while *H is not a DAG* **do**
  $\mathscr{C} \leftarrow \text{STRONGLYCONNECTEDCOMPONENTS}(H)$
  foreach *$S \in \mathscr{C}$ such that $|V(S)| > 1$* **do**
    $L_S \leftarrow \text{BUILDLINEDIGRAPH}(S)$
    if $|V(L_S)| = 0$ **then**
      continue
    $p \leftarrow \text{LINEDIGRAPHPAGERANK}(L_S, k)$
    $C \leftarrow$ the first $\min\{K, |V(L_S)|\}$ arcs after sorting p in non-increasing order
    if $|C| = 1$ **then**
      let e be the only arc in C
      $F \leftarrow F \cup \{e\}$
      remove e from H
      continue
    initialize an undirected graph G_{conf} with vertex set C and no edges
    foreach *unordered pair $\{e_i, e_j\} \subseteq C$* **do**
      $\phi \leftarrow \text{PAIRWISEFEATURES}(H, e_i, e_j, p(e_i), p(e_j))$
      $\widehat{q} \leftarrow \mathscr{M}.\text{PREDICTPROBABILITY}(\phi, 1)$
      if $\widehat{q} \geq \tau$ **then**
        add $\{e_i, e_j\}$ to $E(G_{\text{conf}})$
    $B \leftarrow \text{APPROXIMATEMAXIMUMINDEPENDENTSET}(G_{\text{conf}})$
    if $B = \varnothing$ **then**
      $B \leftarrow \{C[1]\}$
    foreach $e \in B$ **do**
      $F \leftarrow F \cup \{e\}$
      remove e from H
return F

The SCCs are computed at the beginning of each global pass. Each component object is a copy of the corresponding induced subgraph, while removals are applied to the working graph H. Since distinct SCCs have disjoint vertex sets and therefore disjoint internal arc sets, processing the SCCs obtained in one pass does not invalidate candidate arcs belonging to another SCC from that same decomposition. The SCC decomposition is recomputed before the next pass.

3.8 Validity, termination, and computational cost

Termination. Whenever the outer loop is entered, H contains at least one directed cycle and therefore at least one non-trivial SCC after self-loops have been removed. For every processed component, the algorithm removes either the independent-set batch or one fallback candidate. Thus, at least one arc is removed during a cyclic pass. Since the input graph has finitely many arcs, the procedure terminates.

FAS validity. At termination, H is a DAG. Every arc removed from the original graph has been appended to F, so

$$H = G - F.$$

Therefore, F is a valid feedback arc set. This argument establishes acyclicity of the residual graph, but it does not establish minimality or minimum cardinality.

Cost of candidate-pair construction. If

$$r = \min\{K, |E(S)|\},$$

then the algorithm evaluates

$$\binom{r}{2} = \frac{r(r-1)}{2} \tag{28}$$

pairs. With the default $K = 20$, at most 190 pairs are evaluated for one SCC in one pass.

The implementation computes two directed shortest-path queries for every pair. For an unweighted graph, breadth-first search gives each successful or unsuccessful single-source shortest-path search worst-case cost [21, 23]

$$O(|V(H)| + |E(H)|).$$

Consequently, a conservative upper bound for pairwise feature extraction in one component is

$$O\big(r^2\,(|V(H)| + |E(H)|)\big), \tag{29}$$

in addition to neighborhood intersections and model inference.

Line-digraph and PageRank cost. For an SCC S, the line digraph has $|E(S)|$ vertices and

$$\sum_{v \in V(S)} d_S^-(v) d_S^+(v)$$

arcs. With a fixed five PageRank steps, line-digraph construction and propagation have cost

$$O\left(|E(S)| + \sum_{v \in V(S)} d_S^-(v) d_S^+(v)\right), \tag{30}$$

up to implementation-dependent constants. Comparison sorting of all line-graph vertices by score costs [21]

$$O(|E(S)| \log |E(S)|). \tag{31}$$

Amortization mechanism. Suppose that a batch B contains $b = |B|$ arcs. The current line digraph and PageRank vector then support b removals before the next global recomputation. In contrast, the baseline removes one selected arc per SCC in that pass. This is the intended source of the speedup.

The number of passes is not exactly $|F|/\overline{b}$, because SCCs are processed in parallel within a pass, their sizes change after removals, and batch sizes vary. Runtime and solution-quality improvements must therefore be established through the experiments in Section 4 rather than inferred from a simplified closed-form expression.

3.9 Implementation limitations and evidence boundary

The current implementation has several limitations that are important when interpreting its results.

1. **Labels are relative to a heuristic.** A conflict label compares forced pair removal with one run of the implemented PageRankFAS baseline. It does not compare either result with an exact minimum FAS.

2. **Pairwise compatibility does not imply global compatibility.** The conflict graph contains only pairwise predictions. Even if every pair in a selected batch is predicted safe, interactions involving three or more arcs may still increase the final FAS cardinality.

3. **Predictions can be incorrect.** An independent set is free only of conflicts predicted at the selected threshold. It is not guaranteed to be free of true conflicts.

4. **Training and inference candidate policies differ.** Training selects the top ten scored arcs globally across all SCCs, so the generated dataset may contain pairs from different SCCs. Inference selects up to twenty arcs separately inside each SCC and predicts only within-component pairs. This distribution mismatch should be considered when interpreting generalization results.

5. **Synthetic training distribution.** Training graphs contain between 30 and 100 vertices and use edge probabilities between 0.05 and 0.15. Performance on tournaments, sparse Web graphs, or substantially larger SCCs depends on how well the learned pairwise relationships transfer beyond this distribution.

6. **Random graph generation is not fully reproducible.** Although model splitting and Random Forest fitting use fixed random state 42, the current data-generation workers reseed their random-number generators without a recorded deterministic seed. Re-running the generation script can therefore produce a different training dataset.

7. **Approximate independent-set selection.** The NetworkX routine returns an approximation, not necessarily a maximum-cardinality independent set [3, 20]. Consequently, another predicted-conflict-free batch may contain more candidates than the batch selected by the implementation.

8. **No probability calibration.** The Random Forest output is used directly as a thresholded class score. The present pipeline does not test whether a reported value such as 0.70 corresponds to an empirical conflict frequency of 70%.

9. **No quality guarantee relative to PageRankFAS.** ML-Batching-PageRankFAS always returns a valid FAS, but its cardinality can be larger than, equal to, or smaller than the result produced by the baseline on the same input. This trade-off is measured experimentally.

These limitations do not invalidate the batching formulation. They define its current evidence boundary: the approach is a learned heuristic intended to reduce repeated computation while empirically controlling, rather than theoretically eliminating, deterioration in FAS cardinality.

4 Experiments and Discussion

5 Conclusions

Acknowledgments

First and foremost, I would like to thank my supervisors, Dr. Panagiotis Lionakis and Dr. Michalis Mountantonakis, for their continuous guidance, valuable insights, and encouragement throughout this project.

A heartfelt thank you goes to my girlfriend, Eri, and my family—Ieronymos, Athina, Dr. Chrysanthi, and Dr. Aikaterini—for their love, patience, and for standing by me every step of the way.

I am also truly grateful to my friends—Andreas, Antonis, Dimitris, Filippos, Fotis, Giannis, Kostas, Mimis, Panagiotis, and Pavlos—for the memorable moments, constant support, and for keeping spirits high. Finally, thanks to my coworkers—Alexis, Dimitris, Nikos, Nikos, Panayis, Spyros, Spyros—for their great collaboration, and positive energy.

References

[1] A. Baharev, H. Schichl, A. Neumaier, and T. Achterberg, An exact method for the minimum feedback arc set problem, *ACM Journal of Experimental Algorithmics*, 26(1), Article 1.4, 1–28, 2021. `https://doi.org/10.1145/3446429`.

[2] B. Berger and P. W. Shor, Approximation algorithms for the maximum acyclic subgraph problem, in *Proceedings of the First Annual ACM–SIAM Symposium on Discrete Algorithms (SODA)*, pages 236–243, 1990.

[3] R. Boppana and M. M. Halldórsson, Approximating maximum independent sets by excluding subgraphs, *BIT Numerical Mathematics*, 32(2):180–196, 1992. https://doi.org/10.1007/BF01994876.

[4] F. J. Brandenburg and K. Hanauer, Sorting heuristics for the feedback arc set problem, *Technical Report MIP-1104*, University of Passau, Passau, Germany, 2011.

[5] L. Breiman, Random forests, *Machine Learning*, 45(1):5–32, 2001. https://doi.org/10.1023/A:1010933404324.

[6] S. Brin and L. Page, The anatomy of a large-scale hypertextual Web search engine, *Computer Networks and ISDN Systems*, 30(1–7):107–117, 1998. https://doi.org/10.1016/S0169-7552(98)00110-X.

[7] G. Di Battista, P. Eades, R. Tamassia, and I. G. Tollis, *Graph Drawing: Algorithms for the Visualization of Graphs*, Prentice-Hall, Upper Saddle River, NJ, 1999.

[8] P. Eades, X. Lin, and W. F. Smyth, A fast and effective heuristic for the feedback arc set problem, *Information Processing Letters*, 47(6):319–323, 1993. https://doi.org/10.1016/0020-0190(93)90079-O.

[9] V. Geladaris, P. Lionakis, and I. G. Tollis, Effective computation of a feedback arc set using PageRank, *Journal of Graph Algorithms and Applications*, 27(8):737–757, 2023. https://doi.org/10.7155/jgaa.00641.

[10] D. S. Johnson, The NP-completeness column: An ongoing guide, *Journal of Algorithms*, 3(4):381–395, 1982. https://doi.org/10.1016/0196-6774(82)90032-3.

[11] R. M. Karp, Reducibility among combinatorial problems, in R. E. Miller and J. W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103, Plenum Press, New York, 1972. https://doi.org/10.1007/978-1-4684-2001-2_9.

[12] L. Page, S. Brin, R. Motwani, and T. Winograd, The PageRank citation ranking: Bringing order to the Web, Technical Report 1999-66, Stanford InfoLab, Stanford University, 1999. http://ilpubs.stanford.edu:8090/422/.

[13] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, Scikit-learn: Machine learning in Python, *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[14] S. Pemmaraju and S. Skiena, *Computational Discrete Mathematics: Combinatorics and Graph Theory with Mathematica*, Cambridge University Press, Cambridge, UK, 2003.

[15] M. Simpson, V. Srinivasan, and A. Thomo, Efficient computation of feedback arc set at Web scale, *Proceedings of the VLDB Endowment*, 10(3):133–144, 2016. https://doi.org/10.14778/3021924.3021930.

[16] K. Sugiyama, S. Tagawa, and M. Toda, Methods for visual understanding of hierarchical system structures, *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-11(2):109–125, 1981. https://doi.org/10.1109/TSMC.1981.4308636.

[17] R. E. Tarjan, Depth-first search and linear graph algorithms, *SIAM Journal on Computing*, 1(2):146–160, 1972. https://doi.org/10.1137/0201010.

[18] G. Papadakis, *ML-Batching-PageRankFAS: Machine Learning-Enhanced Feedback Arc Set*, software repository, 2026. https://github.com/papadogiorgis/ml-pagerankfas/.

[19] E. N. Gilbert, Random graphs, *The Annals of Mathematical Statistics*, 30(4):1141–1144, 1959. https://doi.org/10.1214/aoms/1177706098.

[20] A. A. Hagberg, D. A. Schult, and P. J. Swart, Exploring network structure, dynamics, and function using NetworkX, in *Proceedings of the 7th Python in Science Conference*, pages 11–15, Pasadena, CA, USA, 2008.

[21] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, Cambridge, MA, 2022. https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/.

[22] NetworkX Developers, *line_graph — NetworkX documentation*. https://networkx.org/documentation/stable/reference/generated/networkx.generators.line.line_graph.html.

[23] NetworkX Developers, *Shortest Paths — NetworkX documentation*. https://networkx.org/documentation/stable/reference/algorithms/shortest_paths.html.

[24] scikit-learn Developers, *compute_class_weight — scikit-learn documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.utils.class_weight.compute_class_weight.html.

[25] A. Niculescu-Mizil and R. Caruana, Predicting good probabilities with supervised learning, in *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, pages 625–632, 2005. https://doi.org/10.1145/1102351.1102430.